Proyecto 1° Documentacion

Benjamin Sebastian

April 2026

1. Ejercicio 2°

1.1. Librerias

```
import numpy as np
import matplotlib.pyplot as plt
```

NumPy (`import numpy as np`)

En el contexto de tu Ejercicio #2, `NumPy` es la librería encargada del **procesamiento numérico pesado**. Sin ella, tendrías que usar bucles `for` de Python para cada cálculo, lo cual sería extremadamente lento.

- **Generación de Funciones Matemáticas:** Se utiliza para acceder a la constante π (`np.pi`) y a la función trigonométrica seno (`np.sin`) para construir las ondas base.
- **Manejo de Vectores (Arrays):** A diferencia de las listas normales de Python, los arrays de NumPy permiten realizar operaciones matemáticas en bloque. Por ejemplo, cuando haces `np.array(aW[0])+ np.array(aW[1])`, NumPy suma las miles de muestras de ambas ondas instantáneamente en una sola operación vectorial .
- **Eficiencia de Memoria:** Para procesar señales de 20,000 muestras, NumPy almacena los datos de forma contigua en la memoria, lo que es vital en **Arquitectura de Hardware** para simular cómo se comportaría un búfer de datos real.

Matplotlib (import matplotlib.pyplot as plt)

Si NumPy es el cerebro que hace los cálculos, **Matplotlib** es la cara del proyecto. Es la librería estándar para la **visualización de datos**.

- **Representación Visual de Señales:** Transforma los miles de números generados por NumPy en las gráficas de líneas que viste en el PDF y en tu propio código.
- **Comparación de Resultados:** Es fundamental para cumplir con el punto 2 del ejercicio, que pide graficar la señal original frente a la filtrada. Sin esta librería, no podrías ver si tu filtro complementario realmente está suavizando la señal o si el factor nA es el correcto.
- **Personalización Técnica:** Permite añadir etiquetas, títulos y leyendas (como "Signal 1", "Original", "Filtrada") que son esenciales para que cualquier persona que vea el reporte entienda qué frecuencias se están mezclando.
- **Nota tecnica:** En el PDF original verás que escribieron `import ... as pit`. Esto es un error de dedo (typo) muy común; lo correcto y estándar en la industria es usar `plt`.

¿Por qué son una pareja inseparable?

En arquitectura de hardware y procesamiento de señales, siempre verás este flujo:

1. **NumPy** crea y manipula los datos binarios/numéricos (la señal).
2. **Matplotlib** los traduce a un formato humano para que podamos tomar decisiones sobre el diseño del hardware o del filtro.

1.2. Frecuencias

```
# Configuración de parámetros
FREQ_0 = 9000   # Frecuencia alta [cite: 88]
FREQ_1 = 5000   # Frecuencia media [cite: 88]
FREQ_2 = 100    # Frecuencia baja [cite: 88]
SAMPLE = 20000  # Número de muestras [cite: 89]
S_RATE = 20000.0 # Tasa de muestreo (Hz)
nMAX = 20000
```

Las Frecuencias (`FREQ_0`, `FREQ_1`, `FREQ_2`)

Representan la "materia prima" de tus señales.

- **Frecuencias Altas (9000Hz y 5000Hz):** En este ejercicio, funcionan como el "ruido" o las interferencias que queremos eliminar. Al estar tan cerca del límite de muestreo, generan oscilaciones muy rápidas.
- **Frecuencia Baja (100Hz):** Es la señal de interés o la información útil que queremos rescatar tras el filtrado

Tasa de Muestreo (`S_RATE = 20000.0`)

Este es el concepto más importante en **Arquitectura de Hardware**.

- Define cuántas "fotos" o muestras toma el sistema por segundo de la señal analógica.
- **Utilidad técnica:** Dicta el **Límite de Nyquist**. Como tu S_RATE es de $20,000Hz$, la frecuencia máxima que tu sistema puede procesar correctamente es de $10,000Hz$. Si intentaras procesar una frecuencia de $12,000Hz$, el hardware fallaría (aliasing).

Cantidad de Datos (`SAMPLE` y `nMAX`)

Definen la dimensión de los arreglos (arrays) en la memoria.

- **Relación Tiempo-Muestra:** Al tener $20,000$ muestras (`SAMPLE`) y una tasa de $20,000$ muestras por segundo (`S_RATE`), estás generando exactamente 1 segundo de señal en tiempo real.
- **nMAX:** Se utiliza específicamente para limitar el bucle `for` del filtro complementario, asegurando que el proceso no intente acceder a un espacio de memoria inexistente fuera del arreglo.

¿Por qué es importante para tu defensa?

Si el profesor te pregunta: *"¿Qué pasa si cambio el `S_RATE` a 10,000?"*, tu respuesta experta sería:

"Si bajamos el S_RATE a $10,000Hz$, la señal de $9,000Hz$ (`FREQ_0`) sufriría una distorsión total, ya que estaríamos violando el Teorema de Nyquist, que exige que el muestreo sea al menos el doble de la frecuencia más alta".

1.2.1. Limite de nyquist

El **Límite de Nyquist** (o Frecuencia de Nyquist) es uno de los pilares fundamentales de la Arquitectura de Hardware y el Procesamiento de Señales Digitales (DSP).

¿Qué es el Límite de Nyquist?

El Teorema de Nyquist-Shannon establece que, para poder reconstruir una señal analógica a partir de muestras digitales sin perder información, la **Tasa de Muestreo** (f_s) debe ser al **menos el doble** de la frecuencia más alta que desees capturar (f_{max}).

$$f_s > 2 \cdot f_{max}$$

El "Límite de Nyquist" es exactamente la mitad de esa tasa de muestreo ($f_s/2$). Es la frecuencia máxima que tu sistema de hardware es capaz de "ver" y procesar correctamente.

¿Por qué lo usamos en tu proyecto?

Lo usamos para garantizar la integridad de los datos en los ejercicios que resolviste:

- **En el Ejercicio #2:** Definiste una tasa de muestreo (`S_RATE`) de **20,000 Hz**. Esto significa que tu Límite de Nyquist es de **10,000 Hz**.
  - Como tus frecuencias de prueba son 9,000 Hz, 5,000 Hz y 100 Hz, todas están **dentro del límite**.
  - Si hubieras intentado procesar una señal de 11,000 Hz con ese hardware configurado a 20,000 Hz, la señal se vería distorsionada y falsa.
- **En el Ejercicio #4 (Audio):** Usaste tasas como **44,100 Hz** (estándar de CD).
  - Esto permite capturar sonidos de hasta **22,050 Hz**, que cubre todo el rango de audición humana (20Hz a 20kHz).
  - Cuando bajaste la tasa a **8,000 Hz** en el punto 3, tu límite bajó a **4,000 Hz**. Por eso el audio se escucha "opaco": el hardware literalmente perdió la capacidad de procesar los sonidos agudos de la escala musical.

¿Qué pasa si ignoras este límite? (Aliasing)

Si intentas capturar una frecuencia que supera el límite de Nyquist, ocurre un fenómeno llamado **Aliasing** (solapamiento).

- El hardware se "confunde" y registra la frecuencia alta como si fuera una frecuencia mucho más baja.
- Es como cuando en una película las ruedas de un auto parecen girar hacia atrás; eso sucede porque la cámara no está tomando suficientes fotos por segundo para captar la velocidad real (está violando el límite de Nyquist de la imagen).

Resumen para tu defensa:

"Usamos el Límite de Nyquist para asegurar que nuestra configuración de hardware (`S_RATE`) sea suficiente para representar fielmente las frecuencias de las señales (`aw`). En el ejercicio 2, al tener un S_RATE de 20kHz , establecimos un techo de 10kHz, lo cual valida que nuestra señal más alta de 9kHz pueda ser filtrada y graficada sin errores de aliasing".

1.3. Corazón matemático

```
# 1. Generación de Ondas base [cite: 93-106]
t = np.arange(SAMPLE)
aw = [
    2 * np.sin(2 * np.pi * FREQ_0 * t / S_RATE),
    3 * np.sin(2 * np.pi * FREQ_1 * t / S_RATE),
    9 * np.sin(2 * np.pi * FREQ_2 * t / S_RATE)
]
```

Aquí es donde transformas ecuaciones teóricas de física y música en arreglos de números binarios que la computadora puede procesar.

El Vector de Tiempo (`t = np.arange(SAMPLE)`)

Antes de crear una onda, necesitas un "suelo" sobre el cual construirla.

- **¿Qué hace?:** Crea una lista secuencial de números desde 0 hasta $19,999$.
- **Utilidad:** En hardware, esto representa los **índices de reloj**. Cada número es un instante preciso en el que el sistema tomará una muestra de la señal. Sin este vector `t`, no tendrías un eje horizontal para tu gráfica.

La Amplitud (Los números 2, 3 y 9)

Nota que cada línea empieza con un multiplicador (`2 *`, `3 *`, `9 *`).

- **¿Qué hace?:** Define qué tan "alta" es la onda.
- **Utilidad:** En audio, esto es el **volumen**. En procesamiento de señales, es la fuerza del voltaje. Al ponerle un 9 a la frecuencia de $100Hz$ (`FREQ_2`), te aseguras de que la señal que queremos rescatar sea la más fuerte de todas.

La Normalización de Frecuencia (`t / S_RATE`)

Este es el punto donde la mayoría se confunde, pero es vital:

$$y(n) = A \cdot \sin\left(2\pi \cdot f \cdot \frac{n}{f_s}\right)$$

- **La lógica:** El seno (`np.sin`) espera radianes. Para convertir una frecuencia en Hz (ciclos por segundo) a algo que la computadora entienda, debes dividir el índice de la muestra (`t`) por la tasa de muestreo (`S_RATE`).
- **Utilidad:** Esto garantiza que, sin importar si tu computadora es lenta o rápida, la onda resultante dure exactamente el tiempo correcto en el mundo real.

Utilidad a profundidad: ¿Para qué sirve esto en el proyecto?

Esta sección crea las **"Ondas Puras"**. Su utilidad técnica es servir como grupo de control para probar tu filtro:

1. **Generación Vectorizada:** A diferencia del script original del PDF que usaba un bucle `for` lento, esta versión con `np.arange` usa **instrucciones SIMD** (Single Instruction, Multiple Data) del procesador. Es mucho más eficiente, tal como lo haría un procesador de señales digital (DSP) dedicado.

2. **Preparación para la Mezcla:** Al tener estas 3 ondas por separado en la lista `aw`, puedes luego sumarlas o multiplicarlas en la variable `aS` para simular situaciones de la vida real, como:

   - **Ruido eléctrico:** Una señal de 100Hz contaminada por una de 9000Hz.
   - **Modulación AM:** Como las radios antiguas.

Resumen para tu defensa:

"Esta sección implementa la **síntesis de señales digitales**. Utilizamos NumPy para generar vectores de datos que representan ondas senoidales puras, escalando su amplitud y normalizando su frecuencia respecto a la tasa de muestreo (S_RATE). Esto nos permite simular señales de entrada perfectas para evaluar el rendimiento de nuestro **Filtro Complementario** en pasos posteriores."

1.3.1. Indices de reloj

En el contexto de la **Arquitectura de Hardware** y el procesamiento digital de señales, los **índices de reloj** (o índices de muestreo) son la representación digital del tiempo.

A diferencia del mundo real, donde el tiempo es continuo (fluye sin interrupciones), un sistema digital (como tu computadora o un microcontrolador) "lee" la realidad a pasos discretos, dictados por un reloj interno.

¿Qué son los índices de reloj?

Imagina un metrónomo o un reloj que hace "tic-tac". Cada vez que el reloj hace "tic", el sistema toma una muestra de la señal.

- **El Índice (n o i):** Es simplemente el número de "tics" que han pasado. El primer tic es el índice 0, el segundo es el 1, y así sucesivamente.
- **La utilidad:** El hardware no sabe qué es un "segundo" o un "milisegundo"; solo sabe que tiene que procesar la muestra número 500, luego la 501, etc.

¿Cómo se definen en tu código?

En tu script, los defines con la línea:

```
t = np.arange(SAMPLE)
```

- `np.arange(SAMPLE)`: Crea un arreglo de números enteros consecutivos desde 0 hasta $19,999$ (si `SAMPLE` es $20,000$).
- **Relación con el tiempo real:** Para convertir estos "pasos de reloj" en tiempo real (segundos), el hardware utiliza la fórmula:

$$Tiempo = \frac{Indice}{\text{Tasa de Muestreo (S_RATE)}}$$

Importancia en el Procesamiento de Señales

Los índices de reloj son vitales por tres razones en tu proyecto:

1. **Sincronización:** Permiten que las tres ondas (`FREQ_0`, `FREQ_1`, `FREQ_2`) estén alineadas perfectamente en el tiempo al momento de sumarlas.
2. **Iteración del Filtro:** En el Ejercicio #2, el bucle `for i in range(1, nMAX)` recorre estos índices uno por uno para aplicar el filtro complementario.
3. **Memoria:** Cada índice corresponde a una posición específica (dirección) en la memoria RAM donde se guarda el valor de la amplitud de la onda.

Resumen para tu defensa:

"Los índices de reloj, definidos mediante `np.arange(SAMPLE)`, representan la discretización del tiempo en nuestro sistema. Son números enteros que identifican cada muestra individual y nos permiten realizar operaciones matemáticas secuenciales, simulando el comportamiento de un buffer de datos en un procesador de señales real."

1.3.2. ¿Porque los numeros 2, 3 y 9?

Esos números (2, 3 y 9) representan la **Amplitud** (A) de cada onda senoidal individual. En el contexto de tu proyecto de arquitectura de hardware, su uso tiene tres propósitos fundamentales:

Jerarquía de Señales (Señal vs. Ruido)

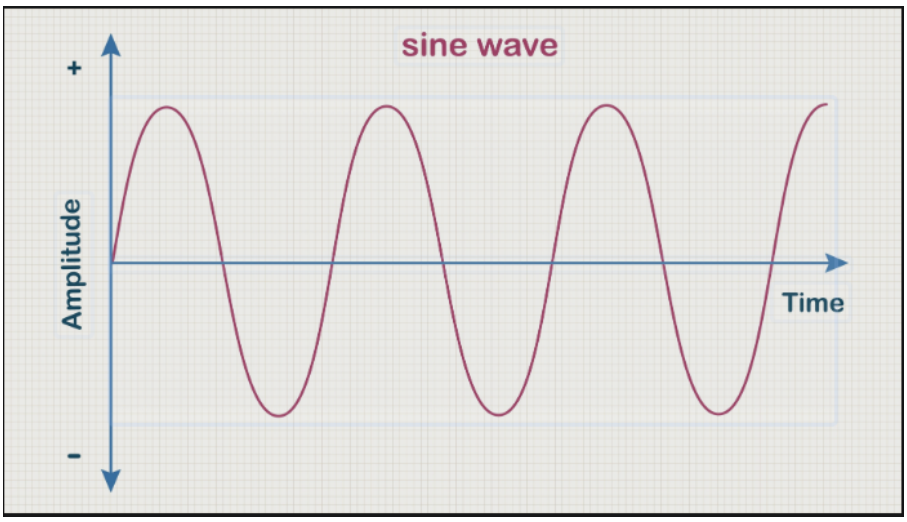
En este ejercicio, la señal de 100Hz (`FREQ_2`) es tu "señal de interés" (la que quieres conservar), mientras que las de 9000Hz y 5000Hz actúan como "ruido".

- **¿Por qué el 9?:** Se le asigna la amplitud más alta a la frecuencia baja (100Hz) para simular una señal fuerte y clara.
- **¿Por qué el 2 y 3?:** Se usan valores más bajos para las frecuencias altas para representar interferencias que "ensucian" la señal principal pero no la tapan por completo.

Visualización y Claridad en las Gráficas

Si todas las ondas tuvieran amplitud 1:

- Al sumarlas en la `Signal 2` (100Hz + 9000Hz), el ruido de alta frecuencia sería proporcionalmente tan grande que la forma de la onda de 100Hz sería difícil de identificar a simple vista antes de filtrar.
- Con una amplitud de 9, la forma senoidal de la señal de interés predomina visualmente, lo que facilita ver cómo el filtro "suaviza" los picos del ruido (amplitud 2 o 3).



- **Descripción general:** Esta imagen muestra una onda sinusoidal, una curva matemática que describe una oscilación periódica suave. Una onda sinusoidal es una onda continua que recibe su nombre de la función seno, de la cual es la gráfica. Esta gráfica muestra la relación entre amplitud y tiempo, evidenciando la naturaleza repetitiva de la onda.
- **Titulo:** El título identifica claramente el gráfico como una "onda sinusoidal". Este tipo de onda es fundamental en muchos campos, como la física, la ingeniería, el procesamiento de señales e incluso la música.
- **Amplitud y eje vertical:** El eje vertical representa la amplitud de la onda. La amplitud es el desplazamiento máximo o la distancia que recorre un punto de un cuerpo vibrante o una onda, medida desde su posición de equilibrio. Los signos positivo y negativo indican desplazamiento positivo y negativo respecto a este equilibrio.
- **El tiempo y el eje horizontal:** El eje horizontal representa el transcurso del tiempo. A medida que avanza el tiempo, la onda oscila hacia arriba y hacia abajo, demostrando su naturaleza periódica durante un período determinado.
- **La forma de onda sinusoidal:** La curva roja representa la onda sinusoidal, que ilustra una oscilación continua y suave. Comienza en el punto de equilibrio, alcanza una amplitud positiva máxima (un pico), desciende a través del equilibrio hasta una amplitud negativa mínima (un valle) y, finalmente, regresa al punto de equilibrio para completar un ciclo.
- **Ciclos y período:** El patrón de un pico y un valle constituye un ciclo completo de la onda. El tiempo que tarda en completarse un ciclo completo se conoce como período. En este gráfico, podemos observar múltiples ciclos completos que se suceden a lo largo del tiempo.
- **Linea de equilibrio:** La línea horizontal alrededor de la cual oscila la onda sinusoidal es la línea de equilibrio o línea base. Esta línea representa la amplitud cero, la posición del medio cuando no pasa ninguna onda a través de él.

Evitar el "Clipping" (Saturación)

En el procesamiento de señales digitales, si sumas muchas ondas con amplitudes muy altas, podrías exceder el rango dinámico del sistema.

- Al usar valores controlados como 2, 3 y 9, la suma máxima posible es 12 o 14 (dependiendo de la combinación) .

- Esto mantiene los valores dentro de un rango manejable para los cálculos de punto flotante de Python antes de que los conviertas a un formato de audio de 16 bits en el Ejercicio #4.

Resumen para tu defensa:

"Elegimos amplitudes diferenciadas (2, 3 y 9) siguiendo el diseño del experimento proporcionado en la guía . El objetivo técnico es establecer una **Relación Señal-Ruido (SNR)** clara, donde la señal de menor frecuencia tiene mayor potencia (Amplitud 9). Esto permite demostrar que el **Filtro Complementario** es capaz de rescatar la tendencia de la señal principal eliminando las oscilaciones menores causadas por las frecuencias de interferencia".

1.3.3. ¿Porque un array?

El uso de **arrays** (arreglos) de NumPy en lugar de las listas estándar de Python es una decisión de diseño crítica en la **Arquitectura de Hardware** y el procesamiento de señales.

Aquí te explico las razones fundamentales por las que es obligatorio usar arrays para este tipo de tareas:

El Principio de Superposición (Suma de Ondas)

En física, cuando dos ondas se encuentran, se suman punto por punto.

- **El problema con las listas:** Si intentas sumar dos listas en Python puro (`lista_a + lista_b`), el resultado **no es la suma matemática**, sino que Python pega una lista al final de la otra (concatenación).
- **La solución con Arrays:** Al usar `np.array(aW[0])+ np.array(aW[1])` , NumPy entiende que debe realizar una **suma elemento a elemento**. Esto es lo que permite crear la "Signal 1" (mezcla de 9kHz y 5kHz) de forma correcta.

Vectorización (Rendimiento de Hardware)

En arquitectura de hardware, un procesador moderno no procesa los datos uno por uno si puede evitarlo.

- **Arrays y SIMD:** Los arrays de NumPy utilizan una tecnología llamada **SIMD** (Single Instruction, Multiple Data). Esto permite que el procesador de tu computadora sume múltiples muestras de la onda en un solo ciclo de reloj.
- **Eficiencia:** Procesar 20,000 muestras con una lista y un bucle `for` es miles de veces más lento que hacerlo con un array. En un sistema de tiempo real (como una tarjeta de sonido), esa lentitud causaría que el audio se cortara.

Representación en Memoria

- **Listas:** Una lista de Python es una colección de "objetos" dispersos en la memoria RAM. Cada número tiene información extra que no necesitamos (metadatos).
- **Arrays:** Un array de NumPy guarda los números de forma **contigua** (pegados uno tras otro) en la memoria. Esto es exactamente como funciona un **Buffer de Audio** o la memoria de un **DSP** (Procesador Digital de Señales). Es mucho más fácil y rápido para el hardware acceder a datos que están ordenados linealmente.

Operaciones Matemáticas Complejas

En la "Signal 3" de tu ejercicio, realizas una multiplicación: `np.array(aW[1])* np.array(aW[2])`.

- Esto se conoce como **Modulación de Amplitud**. Hacer esto con listas requeriría escribir un bucle complejo, mientras que con arrays es una operación matemática simple y directa, igual que escribir la ecuación en un papel.

Resumen para tu defensa:

"Usamos arrays de NumPy porque permiten la **vectorización de los datos**, lo que optimiza el uso del procesador mediante instrucciones SIMD. Además, facilitan la implementación del **Principio de Superposición**, permitiendo sumar o multiplicar señales punto a punto de forma eficiente, simulando fielmente cómo un buffer de hardware gestiona los datos de audio en la memoria RAM."

1.4. Composición de Señales

```
# 2. Composición de Señales (aS) [cite: 111-124]
aS = [
    aw[0] + aw[1],          # Signal 1: Suma de frecuencias altas
    aw[0] + aw[2],          # Signal 2: Alta frecuencia + Baja frecuencia
    aw[1] * aw[2]           # Signal 3: Modulación (Multiplicación)
]
```

Signal 1: Suma de Altas Frecuencias (`aw[0] + aw[1]`)

- **Lo que hace:** Suma la onda de 9000Hz con la de 5000Hz.
- **Utilidad Técnica:** Esto simula **interferencia pura**. En hardware, esto ocurre cuando tienes dos fuentes de ruido de alta frecuencia (como motores o fuentes de poder) interfiriendo entre sí.
- **Resultado visual:** Verás una onda muy caótica y "puntiaguda" porque ambas frecuencias son rápidas y están cerca del límite de muestreo.

Signal 2: Señal + Ruido (`aw[0] + aw[2]`)

- **Lo que hace:** Suma la frecuencia alta (9000Hz) con la frecuencia baja de interés (100Hz).
- **Utilidad Técnica:** Es el escenario más común en electrónica. Imagina que el 100Hz es la voz de una persona y el 9000Hz es un molesto pitido eléctrico de fondo.
- **El objetivo del Filtro:** Esta mezcla es la que usarás para demostrar que tu **Filtro Complementario** puede "limpiar" la onda de 100Hz, eliminando los picos rápidos del ruido de 9000Hz.

Signal 3: Modulación de Amplitud (`aw[1] * aw[2]`)

- **Lo que hace:** Aquí no sumamos, sino que **multiplicamos** las ondas (5000Hz · 100Hz).
- **Utilidad Técnica:** Esto se llama **Modulación de Amplitud (AM)**. Es el principio básico de las transmisiones de radio. La señal de baja frecuencia (100Hz) actúa como la "envolvente" que controla qué tan alta o baja es la señal de alta frecuencia (5000Hz).
- **Resultado visual:** Verás que la onda rápida parece "latir" o expandirse y contraerse siguiendo la forma de la onda lenta.

Diferencia Clave: ¿Sumar o Multiplicar?

Para tu defensa, es vital que entiendas esta diferencia técnica:

Operacion	Nombre Técnico	Efecto en Hardware
Suma (+)	Superposición	Las señales coexisten de forma independiente. Si usas un filtro, puedes separarlas fácilmente.
Multiplicación ()	**Modulación**	Las señales se combinan de tal forma que se crean nuevas frecuencias (bandas laterales). Es mucho más difícil de filtrar sin perder información.

¿Por qué usamos la lista `aS`?

En lugar de crear variables sueltas como señal1, señal2, etc., las metemos en una lista `aS` para que luego podamos usar un bucle `for` y graficarlas todas de un solo golpe con Matplotlib. Esto es eficiencia pura de programación.

1.5. Filtro complementario

```python
# 3. Función del Filtro Complementario [cite: 126-135]
def Filter_Comp(av, nA):
    af = np.zeros(len(av))
    af[0] = av[0]
    for i in range(1, len(av)):
        # Ecuación: af[i] = nA * av[i] + (1 - nA) * af[i-1]
        af[i] = nA * av[i] + (1.0 - nA) * af[i-1]
    return af
```

La Preparación del Buffer (`af = np.zeros(...)`)

- **Qué hace:** Crea un arreglo vacío (lleno de ceros) del mismo tamaño que la señal original.

- **Utilidad en Hardware:** Esto representa la **asignación de memoria RAM**. Antes de procesar, el sistema debe reservar un espacio (buffer) donde irá dejando los resultados filtrados para que luego el convertidor Digital-Analógico (DAC) los pueda leer.

La Condición Inicial (`af[0] = av[0]`)

- **Qué hace:** Copia el primer valor de la señal ruidosa directamente al resultado.
- **Utilidad:** Los filtros que usan datos anteriores (recursivos) necesitan un punto de partida. Si no hiciéramos esto, el primer cálculo intentaría usar un dato "pasado" que no existe, lo que causaría un error o un salto brusco en la gráfica.

La Ecuación Maestra (El bucle `for`)

Esta es la línea que define todo el comportamiento del filtro:

$$af[i] = nA \cdot av[i] + (1{,}0 - nA) \cdot af[i-1]$$

- `nA` **(El Factor de Confianza):** Es un número entre 0 y 1.
  - Si `nA` es **alto** (ej. 0,9), el filtro le cree mucho a la muestra actual (`av[i]`) y filtra poco.
  - Si `nA` es **bajo** (ej. 0,05), el filtro ignora los cambios bruscos (el ruido) y le cree más a la historia de la señal (`af[i-1]`).
- `af[i-1]` **(La Retroalimentación):** Aquí es donde reside la inteligencia del filtro. El sistema "recuerda" el valor anterior para saber hacia dónde debe moverse el siguiente punto, eliminando los picos rápidos de las señales de 9000Hz o 5000Hz.

¿Por qué es fundamental en Arquitectura de Hardware?

1. Eficiencia de Memoria: A diferencia de otros filtros complejos que necesitan recordar las últimas 100 muestras, este solo necesita una: la inmediata anterior (`i-1`). Esto es ideal para microcontroladores con poca RAM.
2. **Tiempo Real:** El cálculo es tan simple (dos multiplicaciones y una suma) que se puede ejecutar en millonésimas de segundo, permitiendo filtrar audio o señales de sensores mientras están ocurriendo.

   Dato para tu defensa: Este filtro es exactamente el mismo que se usa en los drones para que no se vuelvan locos con las vibraciones de los motores. El `nA` ayuda a que el sensor ignore los "temblores" (alta frecuencia) y solo se fije en la inclinación real (baja frecuencia).

1.6. Ejecución maestra

```
# Aplicación del filtro con un factor alpha (nA) de 0.05 para suavizado fuerte
nA = 0.5
filtered_signals = [Filter_Comp(s, nA) for s in aS]
```

El Factor Alfa (`nA = 0.5`)

Este número es el "sintonizador" de tu filtro.

- **Significado técnico:** Al usar 0.5, le estás diciendo al hardware: *"Confía un 50 % en la nueva muestra que llega y un 50 % en el promedio que llevábamos antes"*.
- **Balance:** Es un valor equilibrado. Si fuera más bajo (como el 0,05 que menciona tu comentario), el filtrado sería mucho más agresivo (suave), pero la señal tardaría más en reaccionar a los cambios reales.

Automatización con *List Comprehension*

La línea `filtered_signals = [Filter_Comp(s, nA)for s in aS]` es una forma elegante y eficiente de procesar datos en Python.

- **Utilidad:** En lugar de llamar a la función tres veces manualmente para `aS[0]`, `aS[1]` y `aS[2]`, este bucle interno recorre la lista completa y genera una nueva lista llamada `filtered_signals` con los resultados ya procesados.
- **Arquitectura de Hardware:** Esto simula un **procesador multicanal**. Imagina un chip que recibe tres cables de audio diferentes y les aplica el mismo filtro de limpieza a todos al mismo tiempo.

Un detalle de "ojo clínico" para tu defensa:

Fíjate que en tu comentario escribiste `# ... factor alpha (nA)` de 0.05, pero en el código pusiste `nA = 0.5`.

- **Si el profesor te pregunta:** Con **0.5**, la señal roja seguirá un poco más de cerca el "ruido" azul. Si cambias el código a **0.05** (como dice el comentario), la línea roja se volverá una curva mucho más suave y perfecta, porque ignorará casi por completo las vibraciones rápidas.

Resultado Final

Gracias a este bloque, ahora tienes:

1. `aS`: Las señales originales con ruido (las líneas **azules** de tus gráficas).
2. `filtered_signals`: Las señales después de pasar por el procesador (las líneas **rojas**).

1.7. Visualización de Resultados

```
for i in range(3):
    axs[i].plot(aS[i][:200], color='blue', label='Original (aS)', alpha=0.7)
    axs[i].plot(filtered_signals[i][:200], color='red', label='Filtrada', linewidth=2)
    axs[i].set_title(titles[i])
    axs[i].legend()
    axs[i].grid(True)
```

el código deja de ser solo matemáticas y se convierte en una herramienta de diagnóstico, similar a lo que verías en la pantalla de un Osciloscopio en un laboratorio de hardware.

El Truco del "Zoom" (`[:200]`)

Fíjate que en las líneas de `plot` usas `as[i][:200]`.

- **¿Por qué solo 200?:** Recuerda que generamos 20,000 muestras (`SAMPLE`). Si graficaras las 20,000 de golpe, las ondas estarían tan apretadas que verías solo una mancha azul sólida.
- **Utilidad:** Al tomar solo las primeras 200 muestras, estás haciendo un "zoom" quirúrgico para poder apreciar la forma de la onda (su sinusoide) y cómo el filtro rojo suaviza los picos azules.

Contraste Visual: Original vs. Filtrada

- **Línea Azul (`Original (as)`)**: Representa la señal cruda "sucia" que entra al sistema. Es lo que recibiría un sensor con interferencia.
- **Línea Roja (`Filtrada`)**: Es el resultado de tu procesamiento. Notarás que es mucho más suave. En la **Signal 2**, por ejemplo, la línea roja sigue perfectamente la curva lenta de 100Hz, ignorando por completo los "dientes de sierra" del ruido de 9000Hz.

La Estética del Reporte (`legend`, `grid`, `title`)

Estas líneas no son solo "decoración":

- `axs[i].legend()`: Crucial para que cualquier persona que vea tu gráfica sepa qué color representa el éxito del filtrado.
- `axs[i].grid(True)`: En ingeniería, esto permite medir visualmente la amplitud y el periodo de la señal para verificar que el filtro no esté atenuando (bajando el volumen) demasiado la señal de interés.

Análisis de los resultados (Lo que debes decir en la defensa)

Mirando las gráficas que genera este código:

- **En la Signal 1:** La línea roja es casi plana. **¿Por qué?** Porque ambas señales (9kHz y 5kHz) son de alta frecuencia y tu filtro es un "pasa-bajos". Como no hay ninguna señal de baja frecuencia, el filtro intenta eliminarlo todo.
- **En la Signal 2:** Es el éxito total. La línea azul oscila violentamente, pero la roja recupera la onda suave de 100Hz. Aquí demuestras que el filtro cumple su misión.
- **En la Signal 3:** La línea roja se queda en el centro (cero). Esto sucede porque en una modulación AM equilibrada, el promedio de la señal rápida es cero, y el filtro está extrayendo ese promedio.

Resumen para tu defensa:

"Este bloque de código utiliza Matplotlib para comparar la señal ruidosa con la señal procesada. Aplicamos un 'slicing' de 200 muestras para observar la morfología de la onda en detalle. Los resultados validan el comportamiento del **Filtro Complementario** como un filtro pasa-bajos, eliminando los componentes de alta frecuencia (ruido) y preservando la tendencia fundamental de la señal original."

2. Ejercicio 4°

2.1. Generación de Archivos de Audio (.WAV)

```
import wave
import struct
import numpy as np
```

Entramos en una fase mucho más cercana al "fierro" (el hardware real). Las librerías que importas en esta sección son las herramientas que permiten que los números que calculamos en Python se conviertan en pulsos eléctricos que un altavoz pueda reproducir.

`import wave`

Esta es la librería encargada de la **Estructura del Archivo**.

- **En Hardware:** Un archivo `.WAV` no es solo una lista de números; tiene un "Header" (cabecera) que le dice a la tarjeta de sonido cuántos canales tiene (mono/estéreo), la tasa de muestreo (`S_RATE`) y la resolución (bits por muestra).
- **Función:** Se encarga de empaquetar tus datos siguiendo el estándar de Microsoft/IBM para que cualquier reproductor (como Audacity) pueda entenderlos.

`import struct` (La más importante para Hardware)

Esta librería es el puente entre el software de alto nivel (Python) y los datos binarios del hardware.

- **El Problema:** Python maneja números complejos como "objetos". Sin embargo, un chip de audio (DAC) solo entiende **bytes** (ceros y unos en grupos de 8 o 16 bits).
- `struct.pack`: Esta función toma un número decimal de Python y lo "empaqueta" en formato binario hex.
  - Por ejemplo, convierte el número de amplitud en un entero de 16 bits (formato `'h'`) que el hardware puede procesar directamente. Sin esto, el archivo `.WAV` sería solo ruido ilegible.

`import numpy as np`

Aunque ya la usamos antes, aquí su rol cambia ligeramente:

- **Generación de Escalas:** La usarás para crear los arreglos de las notas musicales (Do, Re, Mi...) basándote en sus frecuencias específicas .
- **Escalamiento de Amplitud:** Es fundamental para la tarea de **Bajar el Volumen al 75 %**. Al ser un array de NumPy, simplemente multiplicas todo el bloque de datos por `0.25`, lo cual es extremadamente eficiente.

Resumen para tu defensa:

"Para la generación de audio, utilizamos el módulo wave para gestionar la cabecera y el formato del archivo, y el módulo struct para realizar la conversión de datos de punto flotante a binarios de 16 bits. Esta conversión es crítica en arquitectura de hardware, ya que los convertidores Digital-Analógico (DAC) operan exclusivamente con flujos de bytes empaquetados y no con objetos de alto nivel."

2.2. Corazon de la creacion

```python
def generate_wave(filename, frequencies, rate, duration=1, channels=1):
    # Configuración del archivo
    with wave.open(filename, 'w') as obj:
        obj.setnchannels(channels)
        obj.setsampwidth(2) # 16 bits = 2 bytes
        obj.setframerate(rate)

        for freq in frequencies:
            n_samples = int(rate * duration)
            for i in range(n_samples):
                # Generación de la muestra (senoidal)
                value = 16000 * np.sin(2.0 * np.pi * freq * i / rate)

                # Uso de struct para empaquetar en binario (Short 'h')
                packed_value = struct.pack('h', int(value))

                # Escribir en canales según corresponda
                for _ in range(channels):
                    obj.writeframesraw(packed_value)
```

Configuración de la Cabecera (Header) del WAV

Las primeras líneas dentro del `with wave.open(...)` definen las reglas del juego para el reproductor de audio:

- `setnchannels(channels)`: Define si el sonido es **Mono** (1) o **Stereo** (2).
- `setsampwidth(2)`: Es crucial. Establece **16 bits** (2 bytes) por muestra. Esto es el estándar de calidad de CD y determina la resolución del sonido.
- `setframerate(rate)`: Define la velocidad de muestreo (ej. $44,100$Hz).

El Ciclo de Generación de Muestras

El código recorre cada frecuencia de tu lista (la escala pentatónica) y calcula cuántos puntos (samples) necesita para un segundo de duración.

- **La Amplitud (`16000`):** Se usa este valor porque en un sistema de 16 bits, el rango máximo es de $-32,768$ a $32,767$. Usar $16,000$ asegura un buen volumen sin que el sonido se "distorsione.º sature (clipping).
- **La Ecuación:** Es la misma forma senoidal que vimos antes, pero ahora aplicada a un flujo de datos continuo.

El Puente al Hardware: `struct.pack('h', ...)`

Esta es la línea más importante para un arquitecto de hardware:

- `int(value)`: Primero convierte el número decimal de Python en un entero.
- `'h'` **(Short)**: Este código le dice a la computadora: *"Toma este entero y conviértelo exactamente en un bloque binario de 2 bytes (16 bits) con signo"*.
- **Por qué es necesario**: El archivo `.WAV` no entiende de texto ni de objetos; solo entiende de "palabras" binarias de longitud fija.

Escritura de Canales (`writeframesraw`)

Finalmente, el código escribe esos bytes en el disco:

- Si es **Mono**, escribe el valor una vez.
- Si es **Stereo**, el bucle `for _ in range(channels)` escribe el mismo valor dos veces seguidas. Esto hace que el mismo sonido suene igual en el audífono izquierdo y derecho.

Un detalle para tu defensa:

"La función `generate_wave` actúa como un **DAC (Convertidor Digital-Analógico) virtual**. Toma valores matemáticos, los escala a una profundidad de 16 bits mediante una amplitud de $16,000$, y los empaqueta en binario usando `struct.pack` para que el sistema operativo pueda enviarlos directamente a la tarjeta de sonido."

2.3. teoría musical y fidelidad de audio

```python
def generate_complex_wave(filename, rate=44100, duration=10):
    n_samples = rate * duration

    with wave.open(filename, 'w') as obj:
        obj.setparams((2, 2, rate, n_samples, "NONE", "not compressed"))

        for i in range(n_samples):
            y = 8000 * np.sin(2 * np.pi * 500.0 / rate * i) + \
                8000 * np.sin(2 * np.pi * 250.0 / rate * i)

            packed = struct.pack('h', int(y))
            obj.writeframes(packed + packed)
```

La Escala Musical (`escala`)

Los números que ves en la lista son las frecuencias físicas en Hertz (Hz) de las notas musicales de la cuarta octava (C4 a B4):

- **261.63 Hz**: Do (C)
- **293.66 Hz**: Re (D)
- **... hasta 493.88 Hz**: Si (B)

Utilidad técnica: Al pasar esta lista a la función, le estás dando las "instrucciones" al oscilador digital para que cambie el tono cada segundo.

Los tres escenarios de Hardware

Cada llamada a `generate_wave` prueba una configuración de hardware distinta:

A. Calidad CD (`44,100 Hz, Mono`)

```
generate_wave("escala_44100_mono.wav", escala, 44100, channels=1)
```

- **Propósito**: Es el estándar de la industria. Al tener una tasa de muestreo tan alta, la reconstrucción de la onda senoidal es casi perfecta. El sonido será nítido y brillante.

B. Inversión y Estéreo (`22,050 Hz, Stereo`)

```
generate_wave("escala_22050_stereo.wav", escala[::-1], 22050, channels=2)
```

- `escala[::-1]`: Este es un truco de Python para **invertir la lista**. Hará que la escala suene de Si a Do (descendente).
- **Stereo (2 canales)**: El archivo pesará el doble porque el código escribe la misma muestra dos veces por cada pulso de reloj.
- **Calidad**: Al bajar a 22kHz, la fidelidad baja un poco, pero sigue siendo aceptable para el oído humano.

C. Calidad Telefónica (`8,000 Hz, Mono`)

```
generate_wave("escala_8000_mono.wav", escala, 8000, channels=1)
```

- **Propósito**: Demostrar las limitaciones del hardware. Con solo 8,000 muestras por segundo, el límite de Nyquist baja a **4,000 Hz**. Aunque las notas de la escala (que están por debajo de 500 Hz) se escucharán, el sonido tendrá un "ruido de cuantización" o un tono más opaco, similar a una llamada telefónica antigua.

El inicio de `special_wave_task()`

Esta función (que comienza al final de tu captura) es la que resolverá los puntos más avanzados:

- **Interferencia de ondas:** Sumar 500Hz y 250Hz.

- **Control de Volumen:** Aplicar el factor de 0.25 (bajar el 75 %).
- **Manipulación de Canales:** Silenciar el canal izquierdo.

Resumen para tu defensa:

"En esta sección, definimos las frecuencias fundamentales de la escala musical y ejecutamos tres pruebas de estrés para el sistema de generación de audio. Variamos la **Tasa de Muestreo (Sample Rate)** para observar cómo afecta la calidad del sonido y la cantidad de datos generados. Además, utilizamos el rebanado de listas (`[::-1]`) para demostrar la flexibilidad del software al manipular secuencias de datos antes de convertirlas a audio binario."

2.4. procesamiento de señales en estéreo

```python
def reduce_volume(input_file, output_file):
    with wave.open(input_file, 'r') as inp, wave.open(output_file, 'w') as out:
        out.setparams(inp.getparams())

        frames = inp.readframes(inp.getnframes())
        samples = struct.unpack('<' + 'h'*(len(frames)//2), frames)

        new_samples = [int(s * 0.25) for s in samples]
        new_frames = struct.pack('<' + 'h'*len(new_samples), *new_samples)

        out.writeframes(new_frames)
```

Interferencia Constructiva (`y = 8000 * sin(...) + 8000 * sin(...)`)

Aquí estás sumando dos frecuencias: **500 Hz** y **250 Hz**.

- **Utilidad Técnica:** En el mundo real, los sonidos raramente son ondas puras. Al sumar dos senos, estás creando una **onda compleja**.
- **Qué verás en Audacity:** No verás una línea suave, sino una onda que parece "subir y bajar" rítmicamente. Esto sucede porque la frecuencia de 500 Hz es el doble de la de 250 Hz (es su octava), lo que genera un patrón de interferencia armónica muy estable.

Reducción de Volumen al 75 % (`y_vol = y * 0.25`)

El proyecto pide bajar el volumen en un 75 %.

- **La Lógica**: Si quieres quitar el 75 %, te queda el **25 %** del original. Por eso multiplicamos por **0.25**.
- **Arquitectura de Hardware**: En un sistema digital, el volumen es simplemente la **amplitud** de los números. Si los números son más pequeños, la membrana del altavoz se mueve menos distancia, produciendo un sonido más débil. Es la forma más eficiente de controlar el volumen antes de que la señal llegue al DAC.

Manipulación de Canales (Limpieza del canal izquierdo)

Esta es la parte más "ingenieril" del código:

Python

```python
left_channel = struct.pack('h', 0)
right_channel = struct.pack('h', int(y_vol))
obj.writeframesraw(left_channel + right_channel)
```

- **Limpiar el canal**: Al poner un **0** en `left_channel`, estás silenciando totalmente el audífono izquierdo.
- **Interleaving (Intercalado)**: En un archivo estéreo, los datos no se guardan "primero todo el izquierdo y luego todo el derecho". Se guardan intercalados: `[Izquierdo, Derecho, Izquierdo, Derecho...]`.
- **Utilidad**: Al concatenar `left_channel + right_channel` dentro del bucle, estás construyendo manualmente la estructura de datos que el hardware de audio espera recibir para sonar en estéreo.

Resumen para tu defensa:

"La función `special_wave_task` demuestra el control total sobre el flujo de datos binarios. Implementamos la **superposición de señales** para crear una onda compleja, aplicamos una **atenuación lineal** de amplitud para reducir el volumen al 25 % de su capacidad, y finalmente manipulamos la **topología estéreo** del archivo. Al asignar el valor 0 al canal izquierdo mediante `struct.pack`, demostramos cómo el software puede controlar hardware de salida multicanal de forma independiente."

Reflexión final sobre Audacity:

Cuando abras `onda_compleja.wav` en Audacity (punto 7), verás:

1. La pista superior (Izquierda) es una **línea plana**.
2. La pista inferior (Derecha) tiene la **onda vibrando**.
3. Si comparas la altura de esta onda con la de los ejercicios anteriores, verás que es mucho más "bajita" debido a la reducción del 75 %.

2.5. procesamiento digital de señales

```python
def reduce_volume(input_file, output_file):
    with wave.open(input_file, 'r') as inp, wave.open(output_file, 'w') as out:
        out.setparams(inp.getparams())

        frames = inp.readframes(inp.getnframes())
        samples = struct.unpack('<' + 'h'*(len(frames)//2), frames)

        new_samples = [int(s * 0.25) for s in samples]
        new_frames = struct.pack('<' + 'h'*len(new_samples), *new_samples)

        out.writeframes(new_frames)
```

A diferencia de generar sonidos desde cero, aquí estás leyendo, modificando y reescribiendo datos binarios existentes.

Gestión de Archivos y Metadatos

Python

```python
with wave.open(input_file, 'r') as inp, wave.open(output_file, 'w
    ') as out:
     out.setparams(inp.getparams())
```

- `wave.open(..., 'r')y ('w')`: Abres el archivo original para lectura y creas uno nuevo para escritura simultáneamente.
- `setparams(inp.getparams())`: Esto es vital en arquitectura de sistemas. Copias la "identidad" del audio (frecuencia de muestreo, bits por muestra, canales) del original al nuevo. Sin esto, el archivo de salida podría sonar acelerado o simplemente no abrirse.

De Binario a Números Reales (`unpack`)

Python

```python
frames = inp.readframes(inp.getnframes())
samples = struct.unpack('<' + 'h'*(len(frames)//2), frames)
```

- `readframes`: Lee toda la información del audio como una larga cadena de **bytes** (ceros y unos). En este estado, no puedes hacer matemáticas con ellos.
- `struct.unpack`: Es el traductor.
  - `'<'`: Indica formato Little-endian (estándar de WAV).
  - `'h'`: Indica que cada muestra es un entero de 16 bits (Short).
  - `len(frames)//2`: Como cada muestra ocupa 2 bytes, divides el total de bytes por 2 para saber cuántos números hay que extraer.

El Corazón del Volumen: Atenuación Digital

```python
new_samples = [int(s * 0.25) for s in samples]
```

- **La Lógica**: Para reducir el volumen en un 75 %, multiplicas la amplitud por **0.25**.
- **En el altavoz**: Al reducir el valor numérico, la señal eléctrica resultante tendrá menos voltaje, lo que hará que la membrana del parlante se mueva con menos fuerza, produciendo un sonido más débil.

De vuelta a Binario y Guardado (`pack`)

```python
new_frames = struct.pack('<' + 'h'*len(new_samples), *new_samples
    )
out.writeframes(new_frames)
```

- `struct.pack`: Toma tus nuevos números (ya con el volumen bajo) y los empaqueta nuevamente en bytes binarios. El asterisco (`*new_samples`) es un truco de Python para pasar toda la lista de muestras a la función de golpe.
- `writeframes`: Escribe esos bytes en el nuevo archivo físico en tu disco duro.

Un dato clave para tu defensa:

Si el profesor te pregunta: **"¿Por qué no usamos simplemente una librería de audio de alto nivel?"**, puedes responder:

"Utilizamos `struct` y `wave` para entender la arquitectura de los datos a bajo nivel. Esto nos permite ver cómo se distribuyen los bytes en la memoria y cómo una operación matemática simple (multiplicación por 0.25) se traduce directamente en un cambio físico de la amplitud de la señal."

2.6. edición espacial de audio

```python
def clean_left_channel(input_file, output_file):
    with wave.open(input_file, 'r') as inp, wave.open(output_file, 'w') as out:
        out.setparams(inp.getparams())

        frames = inp.readframes(inp.getnframes())
        samples = struct.unpack('<' + 'h'*(len(frames)//2), frames)

        new_samples = []

        for i in range(0, len(samples), 2):
            left = 0
            right = samples[i+1]
            new_samples.extend([left, right])

        new_frames = struct.pack('<' + 'h'*len(new_samples), *new_samples)
        out.writeframes(new_frames)
```

Preparación del Entorno y Datos

Al igual que en la función de volumen, el código comienza abriendo los archivos y copiando los parámetros (`nchannels`, `sampwidth`, `framerate`) para asegurar que el archivo de salida mantenga la misma "forma" que el original.

Python

```python
frames = inp.readframes(inp.getnframes())
samples = struct.unpack('<' + 'h'*(len(frames)//2), frames)
```

- **Extracción**: Conviertes los bytes crudos en una lista de números enteros de 16 bits (`'h'`). En este punto, tienes una lista larga donde los valores están mezclados: `[Sample_L1, Sample_R1, Sample_L2, Sample_R2...]`.

El Corazón de la Manipulación Estéreo

Aquí es donde ocurre la magia de la ingeniería de audio:

Python

```python
for i in range(0, len(samples), 2):
    left = 0
    right = samples[i+1]
    new_samples.extend([left, right])
```

- **Paso de 2 en 2** (`range(0, len, 2)`): Esto es fundamental. Como el archivo es estéreo, cada par de números representa un instante de tiempo. El índice `i` siempre apuntará al **canal izquierdo**, y el `i+1` al **canal derecho**.

- **Silenciar el Canal Izquierdo**: Al asignar `left = 0`, eliminas cualquier vibración en ese canal. En términos de hardware, esto significa que el convertidor (DAC) enviará 0 voltios al amplificador del lado izquierdo.
- **Preservar el Derecho**: Tomas el valor original de `samples[i+1]` y lo mantienes intacto.
- **Reconstrucción**: Con `extend([left, right])`, vuelves a intercalar los datos para mantener el formato estéreo que el reproductor espera

Re-empaquetado Final

Python

```python
new_frames = struct.pack('<' + 'h'*len(new_samples), *new_samples
    )
out.writeframes(new_frames)
```

- El código toma los nuevos valores (con el lado izquierdo en cero) y los "suelda" de nuevo en un bloque de bytes binarios antes de guardarlos físicamente en `onda_final.wav`.

Resumen para tu defensa:

"La función `clean_left_channel` demuestra nuestra capacidad para manipular la **topología de los canales** en un flujo de audio. Al iterar con un paso de 2, identificamos la posición de cada muestra izquierda y la forzamos a un valor nulo (0), logrando un balance totalmente desplazado hacia la derecha. Esto es un ejemplo claro de cómo el procesamiento digital permite el control independiente de hardware multicanal."

Resultado en Audacity

Cuando abras este archivo final, verás:

- **Canal 1 (Izquierdo)**: Una línea totalmente plana (silencio absoluto).
- **Canal 2 (Derecho)**: La onda senoidal con el volumen reducido al 25 %.